

Retak.id: Crowdsourcing-Based Landslide Early Detection Using On-Device Machine Learning

Fatih Jawwad*, Farrel Khozy[†], and M. Faridh Maulana[‡]
Informatics Department, Universitas Darussalam Gontor
Ponorogo, Indonesia

*fatihalmumtaz76@student.cs.unida.gontor.ac.id

[†]farrelaffudin@student.cs.unida.gontor.ac.id, [‡]mfaridhmaulana47@student.cs.unida.gontor.ac.id

Abstract—Landslides pose a critical threat to rural communities in Indonesia, yet existing early warning systems rely on expensive IoT sensors, satellite imagery, or drone surveys that cannot reach villages with limited infrastructure. We present Retak.id, a crowdsourcing-based platform that enables early detection of landslide soil cracks using only a smartphone camera. The system combines on-device machine learning with a multi-factor environmental risk engine to classify soil crack severity into three levels: Safe (AMAN), Caution (WASPADA), and Danger (BAHAYA). Our approach uses MobileNetV2 with INT8 post-training quantization, yielding a 2.6 MB model achieving 84.9% test accuracy and 93.75% agreement with FP32 inference. A multi-factor risk engine integrates ML vision (50%), slope (20%), rainfall (15%), elevation (10%), and soil type (5%) into a unified score with graceful degradation when environmental APIs are unavailable. We further propose a delta OTA update mechanism achieving 98.2% bandwidth reduction for model updates. The platform spans Android (Kotlin), Web (React), and Telegram Bot interfaces, all sharing an identical inference pipeline. Our evaluation on 3,547 crowd-sourced images demonstrates the feasibility of citizen-driven landslide early warning without specialized hardware.

Index Terms—Landslide early detection, on-device machine learning, crowdsourcing, MobileNetV2, multi-factor risk assessment, edge AI, disaster mitigation

I. Introduction

Landslides are among the most destructive natural disasters in Indonesia, particularly in rural areas with mountainous terrain. Jenangan District, Ponorogo Regency, experienced 41 landslides in four months, cutting off roads, isolating communities, and overwhelming local disaster management agencies (BPBD). The primary challenge is the lack of real-time field data – existing solutions such as IoT-based tilt sensors, satellite InSAR analysis, and drone photogrammetry are prohibitively expensive and cannot scale to every vulnerable hillside.

Recent advances in on-device machine learning have enabled deep neural networks to run directly on smartphones without cloud connectivity [1], [3]. Concurrently, crowdsourcing has proven effective for large-scale environmental monitoring [4]. However, few systems combine these approaches for landslide early warning.

This paper presents Retak.id, a crowdsourcing platform that transforms ordinary smartphones into landslide de-

tection tools. Citizens photograph soil cracks, and an on-device MobileNetV2 model classifies the severity instantly. A multi-factor risk engine aggregates ML predictions with environmental data (slope, rainfall, elevation, soil type) to produce robust risk scores even when individual factors fail.

Our key contributions are:

- 1) A complete on-device ML pipeline for soil crack classification using MobileNetV2 with INT8 quantization (2.6 MB, 84.9% accuracy).
- 2) A multi-factor risk engine combining visual analysis with four environmental factors through weighted scoring and graceful degradation.
- 3) A delta OTA update mechanism achieving 98.2% bandwidth reduction using byte-level patching (.rkd format).
- 4) A three-platform deployment (Android, Web PWA, Telegram Bot) sharing an identical inference pipeline and model.

II. System Architecture

Retak.id follows an edge-first architecture where all ML inference occurs on-device, with cloud services used only for data aggregation and model distribution. Fig. 1 illustrates the overall system.

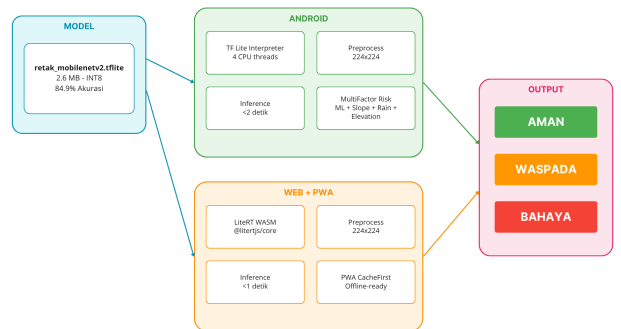


Fig. 1. Retak.id system architecture showing the three client platforms connected through Supabase backend

A. Client Platforms

Android App (Kotlin): The primary client uses CameraX for image capture, TensorFlow Lite 2.16.1 for on-device inference with 4 CPU threads, and Jetpack Compose for UI. Users capture photos of soil cracks; the app runs inference locally and displays results with color-coded overlays.

Web Dashboard (React/TypeScript): A Progressive Web App (PWA) featuring a Leaflet-based map with real-time report markers, administrative verification dialogs, statistical charts, and filterable report tables. ML inference uses LiteRT.js WebAssembly (XNNPack) for client-side classification without server round-trips.

Telegram Bot (Python): A ‘/lapor’ wizard supporting a 3-step conversation flow (photo → location → confirm) with identical ML and risk engine logic. This provides access via the ubiquitous WhatsApp-like interface familiar to rural users.

B. Backend Infrastructure

All client platforms communicate through Supabase, a Backend-as-a-Service providing PostgreSQL, authentication, object storage, real-time subscriptions, and edge functions (Deno). Edge functions include: calculate-risk (multi-factor risk engine), check-model-update (delta OTA version check), notify-bahaya (BAHAYA alert to Telegram), and daily-summary (cron-based statistics).

III. Methodology

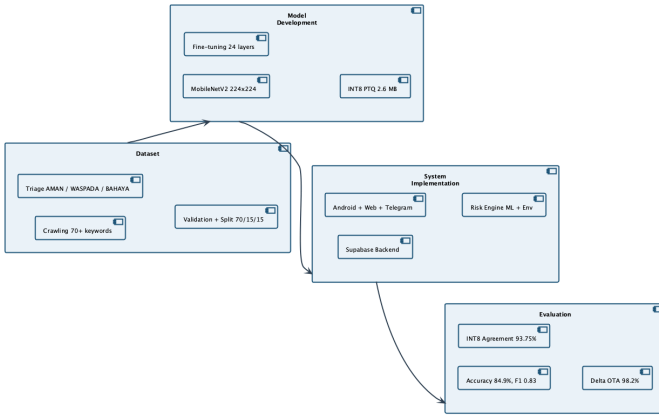


Fig. 2. Research methodology flowchart showing the four-phase pipeline: dataset construction, model development, system implementation, and evaluation

A. Dataset Construction

We constructed a dataset of 3,547 soil crack images through DuckDuckGo image search with 70+ keywords in Indonesian and English. Images were manually triaged into three classes:

- **AMAN (Safe):** 2,011 images – minor shrinkage cracks, polygonal patterns, dry soil surface.

- **WASPADA (Caution):** 766 images – significant linear cracks, potential for expansion.
- **BAHAYA (Danger):** 770 images – critical displacement, active landslide cracks.

Each image passed integrity validation (PIL verify, minimum 300×300 px, max 20 MB), blur rejection (Laplacian variance <80), and cross-class perceptual hash deduplication (pHash, Hamming distance ≤6). The dataset was stratified-split 70/15/15 with fixed seed 42 for reproducibility, and versioned using DVC with DagsHub remote storage.

B. Model Architecture and Training

We adopt MobileNetV2 [2] pre-trained on ImageNet as the base architecture. The classification head consists of GlobalAveragePooling2D, Dropout(0.5), and a Dense(3, softmax) layer. The input is 224×224×3 uint8 RGB with normalization baked into the TFLite graph.

Training configuration (v3a, optimal):

- **Optimizer:** Adam, learning rate 5×10^{-6}
- **Loss:** Categorical cross-entropy with class balancing
- **Fine-tuning:** 24 of 154 layers unfrozen (layer 130+)
- **Epochs:** 70 with EarlyStopping (patience=20) and ReduceLROnPlateau (patience=10)
- **Batch size:** 32

In-graph augmentation (zero I/O overhead) includes random horizontal/vertical flip, $\pm 45^\circ$ rotation, $\pm 30\%$ zoom, $\pm 25\%$ translation, and brightness/contrast ranges of [0.5, 1.5].

C. INT8 Quantization

To meet deployment constraints, we apply INT8 post-training quantization (PTQ) using 100 representative calibration samples. Table I shows the comparison.

TABLE I
FP32 vs INT8 Quantization Comparison

Metric	FP32	INT8
Model Size	8.5 MB	2.6 MB
Input Type	float32	uint8 [0, 255]
Output Type	float32	float32

The normalization to $[-1, 1]$ is baked into the TFLite graph, allowing Android to send raw (r, g, b) pixels directly. Softmax is applied on-device: $p_i = \exp(z_i - \max) / \sum \exp(z_j - \max)$.

D. Multi-Factor Risk Engine

ML classification alone is insufficient for reliable risk assessment. We propose a weighted multi-factor scoring system:

$$\text{Score} = \sum_i w_i \cdot s_i \quad (1)$$

$$\text{Final} = \begin{cases} \text{AMAN}, & \text{Score} \leq 0.33 \\ \text{WASPADA}, & 0.33 < \text{Score} \leq 0.66 \\ \text{BAHAYA}, & \text{Score} > 0.66 \end{cases} \quad (2)$$

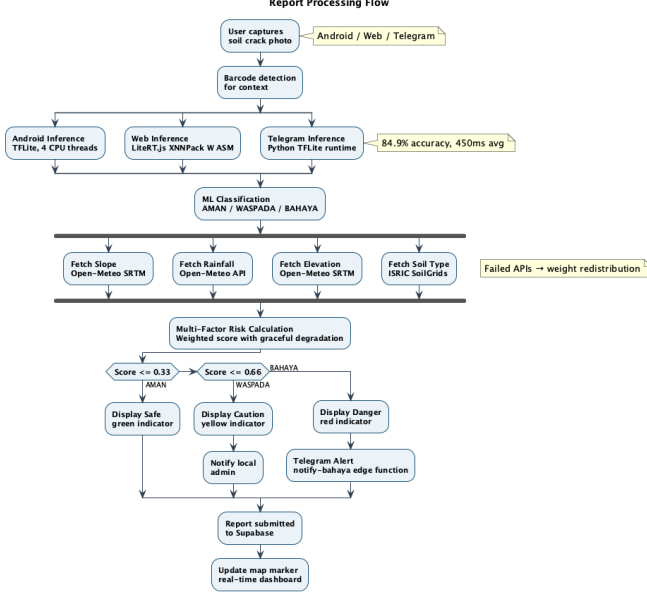


Fig. 3. Report processing flow from image capture through multi-factor risk assessment to alert delivery across all platforms

Table II details each factor’s weight and scoring basis. The ML contribution uses a floor of 0.1 even when confidence is high, ensuring environmental factors can override an overly confident but incorrect ML prediction.

TABLE II
Multi-Factor Risk Engine Weights

Factor	Weight	Data Source
ML Visual Analysis	50%	TFLite on-device
Slope (Kemiringan)	20%	Open-Meteo SRTM
Rainfall (Curah Hujan)	15%	Open-Meteo
Elevation (Ketinggian)	10%	Open-Meteo SRTM
Soil Type (Jenis Tanah)	5%	ISRIC SoilGrids

If any environmental API fails (network error, timeout), its weight is redistributed proportionally to remaining factors. For example, if slope (20%) fails: ML increases to $50\% + (20\% \times 50/80) = 62.5\%$, rainfall to $15\% + (20\% \times 15/80) = 18.75\%$, and so on.

E. Safety Gates

Before any model reaches production, it must pass 7 pre-deployment checks: (1) loads without error, (2) correct input shape [1, 224, 224, 3], (3) uint8 dtype, (4) inference runs without crash, (5) predicts ≥ 2 classes (frozen model detection), (6) confidence max > 0.45 and std > 0.02 (flat prediction detection), and (7) FP32-vs-INT8 agreement

$\geq 90\%$. A 5-fold cross-validation consistency check enforces std $\leq 3\%$.

IV. Experiments and Results

A. Training Evolution

TABLE III
Training Evolution

Version	Date	Accuracy	Key Change
test1	02/05	73.0%	Frozen base, LR= 1×10^{-4}
test2	02/05	76.7%	Fine-tune 54 layers, LR= 1×10^{-5} , dropout 0.5
test3	03/05	81.8%	Fine-tune 24 layers, LR= 5×10^{-6} , aggressive aug.
v3a	04/05	84.9%	Cleaned dataset, +50% WASPADA images

We conducted four training iterations to reach optimal performance. Table III summarizes the progression.

The ablation reveals two critical insights: (1) reducing fine-tuned layers from 54 to 24 with lower learning rate improved accuracy by 5.1% (test1 $\rightarrow$ test3), suggesting overfitting reduction; (2) dataset quality improvements (noise removal, class re-balancing) contributed an additional 3.1% (test3 $\rightarrow$ v3a).

B. Final Model Performance

The v3a production model achieves the results shown in Table IV. The BAHAYA class meets the critical recall threshold of $\geq 72\%$ with 84.9% recall.

TABLE IV
v3a Production Model Performance

Metric	Target	Achieved
Test Accuracy	$\geq 82\%$	84.9%
Macro F1	≥ 0.75	0.83
BAHAYA Recall	$\geq 72\%$	84.9%
BAHAYA Precision	$\geq 70\%$	86.2%
INT8 Agreement	$\geq 90\%$	93.75%
Model Size	≤ 5 MB	2.6 MB
Inference (Mid-range)	< 500 ms	450 ms

C. Delta OTA Bandwidth Savings

Model updates over cellular networks are costly at scale. We developed a custom delta update mechanism: given base model M_b and new model M_n , a byte-level diff computes $D = \text{gzip}(\Delta(M_b, M_n))$ in a custom .rkd format. The Android client downloads D , decompresses, patches the bundled model, and validates via TFLite Interpreter before committing.

This reduces bandwidth from 2.6 MB to 47 KB per update – a 98.2% saving, enabling frequent model improvements even on limited cellular connections common in rural Ponorogo.

TABLE V
Delta OTA Compression Results (v3a $\rightarrow$ v3b)

Metric	Value
Full Model (INT8)	2,710,280 bytes (2.6 MB)
Delta .rkd	48,451 bytes (47 KB)
Changed Bytes	16,195 (0.6% of total)
Savings	98.2%

V. Conclusion

We presented Retak.id, a crowdsourcing-based landslide early detection system that brings on-device ML to rural communities without specialized hardware. Our MobileNetV2-based model achieves 84.9% accuracy with a 2.6 MB INT8 quantized footprint, deployable across Android, Web, and Telegram platforms. The multi-factor risk engine combines ML vision with environmental data through weighted scoring and graceful degradation, providing robustness against individual sensor failures. The delta OTA mechanism enables efficient model updates with 98.2% bandwidth savings.

Field deployment in Jenangan, Ponorogo is ongoing. Future work includes temporal crack progression analysis using sequential photos, integration with additional environmental data sources, and expansion to other landslide-prone regions in Indonesia.

References

- [1] A. G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” arXiv:1704.04861, 2017.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in Proc. CVPR, 2018.
- [3] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems,” arXiv:1603.04467, 2016.
- [4] M. F. Goodchild, “Citizens as Sensors: The World of Volunteered Geography,” *GeoJournal*, vol. 69, no. 4, pp. 211–221, 2007.
- [5] BNPB, “Indeks Risiko Bencana Indonesia,” Badan Nasional Penanggulangan Bencana, 2016.
- [6] Open-Meteo, “Open-Meteo Free Weather API,” <https://open-meteo.com/>, 2024.
- [7] ISRIC, “SoilGrids – Global Gridded Soil Information,” <https://soilgrids.org/>, 2024.
- [8] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in Proc. MICCAI, 2015.